

멀티미디어개론 3주 1차시

차시명	텍스트의 개요 및 저장
-----	--------------

Lesson 01

텍스트의개요

1) 개념과 특징



텍스트(Text)

여러 문장이 모여 만들어진 문장의 집합(문장)

멀티미디어에서의 텍스트

사람이 이해할 수 있게 인공적으로 만든
2차원 형태의 미디어(인공, 단일미디어)



1) 개념과 특징



➔ 멀티미디어에서의 텍스트

▶ 문자의 배열 형태로 나타남

예 문자, 기호, 단어, 구, 문장, 다이어그램, 도표, 인터넷 주소 등(URL, URI)

▶ 멀티미디어 데이터 중 가장 많이 사용되며 가장 수월하게 조작할 수 있는 데이터임(많이, 수월)

▶ 다른 종류의 멀티미디어 데이터보다 기억 용량을 적게 차지함(적게)

▶ 사용하는 언어에 제한을 받는다는 특징이 있음(제한)



1) 개념과 특징



➔ 멀티미디어에서의 텍스트



2) 문서편집기와 워드프로세서



➔ 문서편집기

문서편집기(Text Editor)

단순히 텍스트 파일을 읽고 간단히 **편집**하고 저장하는 데 사용하는 프로그램



대표적인 문서편집기는 윈도우의 **메모장(Notepad)**

- ▶ 확장자는 .txt이며, **태그(Tag)**나 **스타일(Style)** 같은 포맷을 가지지 않음
- ▶ 주로 프로그래밍 소스코드를 작성할 때 많이 사용됨(**복사**)



2) 문서편집기와 워드프로세서



➔ 문서편집기

워드프로세서

문서를 작성 · 편집 · 저장 · 인쇄할 때 사용하는 프로그램



대표적인 워드프로세서 프로그램은 **MS 워드**와 **한글**

- ▶ 기본 텍스트 파일을 저장할 수 있을 뿐만 아니라 필요에 따라 **다양한 저장 형식**을 지정할 수 있음(**pdf 파일**)

- ▶ 구체적인 **파일 포맷**과 **제어 문자**도 포함함(**형식**)



2) 문서편집기와 워드프로세서



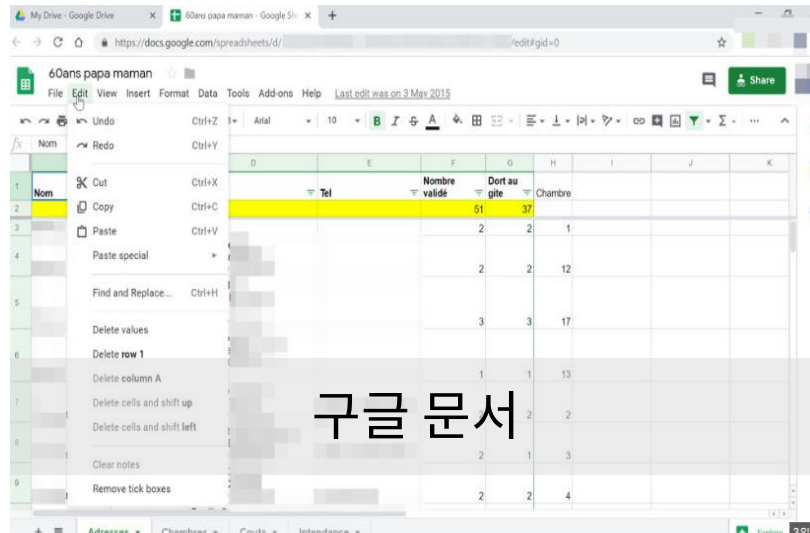
➔ 문서편집기와 워드프로세서의 진화

온라인 워드프로세서 프로그램

- 네이버 워드
- 구글 문서(동기화)

오픈오피스

- 무료로 사용할 수 있는
공개형 워드프로세서
 - 에스메모(SMemo)



2) 문서편집기와 워드프로세서



➔ 문서편집기와 워드프로세서 비교

	문서편집기	워드프로세서
특징	■ 어떠한 종류의 저장 형식도 지원하지 않음 ■ 워드프로세서에 비해 문서 작성 기능이 제한적임 - 글자 크기, 폰트 글자 색깔 등을 바꿀 수 없음 ■ 대부분의 문서편집기와 호환이 잘됨 - 불특정 다수를 대상으로 하는 문서를 작성할 때 사용함 ■ 프로그래밍 소스코드를 작성할 때 사용됨(간단, 복사)	■ 필요에 따라 다양한 형식으로 저장 가능함 ■ 문서편집기보다 다양한 기능을 가짐 - 글자 속성, 문단 속성, 다양한 개체 삽입, 사전 기능, 문법 교정 기능 등(형식) ■ 다른 워드프로세서와 호환성이 낮음 - 한글로 작성한 파일을 MS 워드로 불러오지 못하거나 MS 워드로 작성한 파일을 한글로 불러오지 못함
프로그램 예	윈도우의 메모장, 맥킨토시의 텍스트에디터	한글, MS 워드



2) 문서편집기와 워드프로세서



➔ 음성으로 텍스트 입력하기

음성으로 텍스트 입력

- 텍스트를 입력할 때 키보드나 스캐너를 사용하는 방법 외에도 **음성 인식(Voice/Speech Recognition)** 기능을 이용하는 방법이 있음
- 아이폰 혹은 안드로이드 메모장에서 가상 키보드의 마이크 버튼을 누른 후 입력할 내용을 말하고 [완료] 버튼을 누르면 말 한 내용이 텍스트로 입력됨
- 텍스트를 음성으로 읽어주는 기능도 있음
 - 아이폰과 안드로이드폰에서 자동 텍스트 말하기를 활성화하면 명령과 텍스트를 모두 음성으로 읽어 줌
 - **TTS(Text-to-Speech)** 기능 : 음성을 텍스트로 만들어주는 기능도 있음



3) 텍스트의 표현



→ 코드 시스템

코드 시스템(Code System)

컴퓨터에서 문자를 사용하기 위해 **이진코드**를 일정한 규칙에 따라 각 문자에 할당하는 것(**약속**)

각각의 문자를 컴퓨터에 저장하고 컴퓨터 내부에서 문자들을 구분하여 사용하기 위해 사용됨

컴퓨터 내부에서 모든 문자는 이진코드로 **인코딩(Encoding)**됨
(**부호화**)



복호화하여 본래의 문자나 기호로 변환하여 화면과 프린터에 출력함



3) 텍스트의 표현



→ 코드 시스템

아스키(ASCII)
코드의 문자 'A'

- 이진코드 '1000001'로 **인코딩** 됨(**부호화**)
- 코드값: 65

코드 시스템은 언어에 따라 다름

알파벳 사용권

8비트(**1바이트**) 코드

한자를 사용하는 동양권

16비트(**2바이트**) 코드 사용



3) 텍스트의 표현

→ 표준 코드 시스템

1963년

- 아스키 코드(ASCII)
 - American Standard Code for Information Interchange Code(7bit)

1964년

- EBCDIC(1바이트)
 - Extended Binary Coded Decimal for Interchange Code

1995년

- 유니코드(Unicode)
 - 각 나라의 코드 시스템을 하나로 통합하고 표준으로 제정되어 현재 사용되고 있음(2바이트)

3) 텍스트의 표현

➔ 아스키 코드

아스키 코드(ASCII Code)

세계적으로 널리 사용되고 있는 **코드 체계**로 데이터 처리 및 통신시스템에서 상호간의 **정보 교환용**으로 사용됨(**문자 전송, 패리티 비트**)

▶ 한 문자를 표현하기 위해 **7비트**를 사용함

➡ 표현할 수 있는 문자의 수는 2^7 으로 총 128자를 표현함

3) 텍스트의 표현



➔ 아스키 코드

10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자
0	0x00	NULL	32	0x20	Sp	64	0x40	@	96	0x60	`
1	0x01	SOH	33	0x21	!	65	0x41	A	97	0x61	a
2	0x02	STX	34	0x22	"	66	0x42	B	98	0x62	b
3	0x03	ETX	35	0x23	#	67	0x43	C	99	0x63	c
4	0x04	EOT	36	0x24	\$	68	0x44	D	100	0x64	d
5	0x05	ENQ	37	0x25	%	69	0x45	E	101	0x65	e
6	0x06	ACK	38	0x26	&	70	0x46	F	102	0x66	f
7	0x07	BEL	39	0x27	'	71	0x47	G	103	0x67	g
8	0x08	BS	40	0x28	(	72	0x48	H	104	0x68	h
9	0x09	HT	41	0x29	)	73	0x49	I	105	0x69	i
10	0x0A	₩n	42	0x2A	*	74	0x4A	J	106	0x6A	j
11	0x0B	VT	43	0x2B	+	75	0x4B	K	107	0x6B	k
12	0x0C	FF	44	0x2C	,	76	0x4C	L	108	0x6C	l
13	0x0D	₩r	45	0x2D	-	77	0x4D	M	109	0x6D	m
14	0x0E	SO	46	0x2E	.	78	0x4E	N	110	0x6E	n
15	0x0F	SI	47	0x2F	/	79	0x4F	O	111	0x6F	o
16	0x10	DLE	48	0x30	0	80	0x50	P	112	0x70	p
17	0x11	DC1	49	0x31	1	81	0x51	Q	113	0x71	q
18	0x12	DC2	50	0x32	2	82	0x52	R	114	0x72	r
19	0x13	DC3	51	0x33	3	83	0x53	S	115	0x73	s
20	0x14	DC4	52	0x34	4	84	0x54	T	116	0x74	t
21	0x15	NAK	53	0x35	5	85	0x55	U	117	0x75	u
22	0x16	SYN	54	0x36	6	86	0x56	V	118	0x76	v
23	0x17	ETB	55	0x37	7	87	0x57	W	119	0x77	w
24	0x18	CAN	56	0x38	8	88	0x58	X	120	0x78	x
25	0x19	EM	57	0x39	9	89	0x59	Y	121	0x79	y
26	0x1A	SUB	58	0x3A	:	90	0x5A	Z	122	0x7A	z
27	0x1B	ESC	59	0x3B	;	91	0x5B	[	123	0x7B	{
28	0x1C	FS	60	0x3C	<	92	0x5C	₩	124	0x7C	
29	0x1D	GS	61	0x3D	=	93	0x5D	]	125	0x7D	}
30	0x1E	RS	62	0x3E	>	94	0x5E	^	126	0x7E	~
31	0x1F	US	63	0x3F	?	95	0x5F	_	127	0x7F	DEL



3) 텍스트의 표현



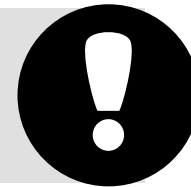
➔ 아스키 코드의 패리티 비트

- ▶ 컴퓨터 환경에서는 일반적으로 8비트를 사용하기 때문에 아스키 코드를 8비트로 구성하여 사용함
- ▶ 공백으로 남는 나머지 1비트는 오류 검출을 목적으로 하는 **패리티(Parity) 비트**로 활용함(**문자 전송**)

짝수(**Even**) 패리티 비트

홀수(**Odd**) 패리티 비트

패리티 비트는 오류는 쉽게 검출할 수 있지만
해결(**정정**)은 불가능함(**짝수 오류, 해밍 코드**)



3) 텍스트의 표현



➔ 아스키 코드의 패리티 비트

▮ 짝수 패리티 비트 사용 시 오류 검출 방법

오리지널 데이터

0 1 1 0 0 0 0 0



전송 데이터

0 1 1 0 0 1 0 0

전송 데이터 8비트에서 1의 개수가 홀수 이므로 오류가 검출됨 (동시에 두 개 발생)



3) 텍스트의 표현



➔ 아스키 코드의 패리티 비트

짝수 패리티 비트와 홀수 패리티 비트

오리지널 데이터	짝수(Even) 패리티	홀수(Odd) 패리티
00000000	0	1
01011011	1	0
01010101	0	1
11111111	0	1
10000000	1	0
01001001	1	0



3) 텍스트의 표현



➔ 확장 아스키 코드(Extended ASCII)

독일어, 불어 같이 영어의 알파벳 외에 **점**이 있는 문자

물결 표시 문자를 사용하는 나라

“ 아스키 코드 대신
확장 아스키 코드를 사용함 ”

▶ 패리티 비트를 사용하지 않고 **8비트**를 모두 문자를 표현하는 데 사용함

➔ 표현할 수 있는 문자의 수는 2^8 , 총 256개의 문자를 표현할 수 있음



3) 텍스트의 표현



➔ EBCDIC(엡시딕)

- ▶ **8비트**를 사용하므로 표현할 수 있는 문자 수는 2^8 총 256개임

상위 4비트
(존 비트, **Zone** Bit)

하위 4비트
(디지트 비트, **Digit** Bit)

- ▶ 각 영역에서 읽은 비트는 **코드 테이블**에 대응시켜 해당 코드가 정의하는 문자를 알아냄(**부호화/복호화**)

근래에 아스키 코드가 주로 사용됨에 따라
많이 사용되지 않음(**중급 컴퓨터**)

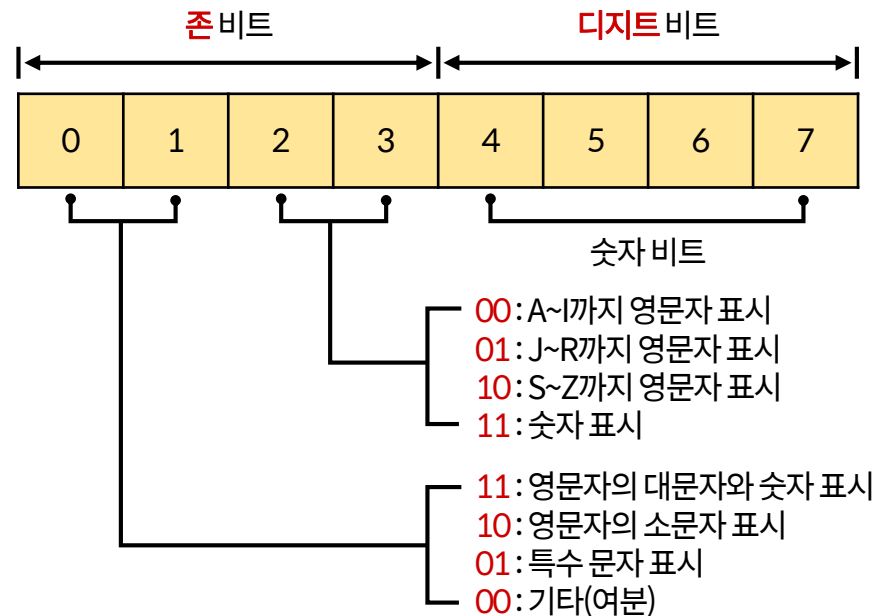


3) 텍스트의 표현

➔ EBCDIC의 코드 테이블과 코드 구조

EBCDIC character codes

	1st hex digit															
2nd hex digit	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	DEL	DS		SP	&	.									0
1	SOH	DC1	SOS				/		a	j	0	0	A	J	0	1
2	STX	DC2	FS	SYN					b	k	s	0	B	K	S	2
3	ETX	TM							c	l	t	0	C	L	T	3
4	PF	RES	BYP	PN					d	m	u	0	D	M	Y	4
5	HT	NL	LF	RS					e	n	v	0	E	N	V	5
6	LC	BS	ETB	UC					f	o	w	0	F	O	W	6
7	DEL	IL	ESC	EOT					g	p	x	0	G	P	X	7
8		CAN							h	q	y	0	H	Q	Y	8
9		EM							i	r	z		I	R	Z	9
A	SMM	CC	SM		CCENT	!		:								
B	VT	CUI	CU2	CU3		\$	.	#								
C	FF	IFS		DC4	<	*	%	@								
D	CR	IGS	EMQ	NAK	(	)	_	'								
E	SO	IRS	ACK		+	:	>	=								
F	SI	IUS	BEL	SUB		..	?	"								



3) 텍스트의 표현



➔ 한글 코드

- ▶ 한글은 8비트로 코드 체계를 표현할 수 없어 **별도의 코드 시스템**이 필요함
- ▶ 글자를 초성, 중성, 종성으로 구분하여 표현하면 조합이 가능한 전체 글자수는 $19 \times 21 \times 28 = 11,179$ 가 됨 **(2바이트)**

조합형 코드

초성, 중성, 종성에 대한
각각의 코드를 저장하고
실제로 출력할 때 **조합**함

완성형 코드

초성, 중성, 종성의
낱글자들을 미리 조합하여
코드로 **완성**시켜 놓은 것 **(사용)**

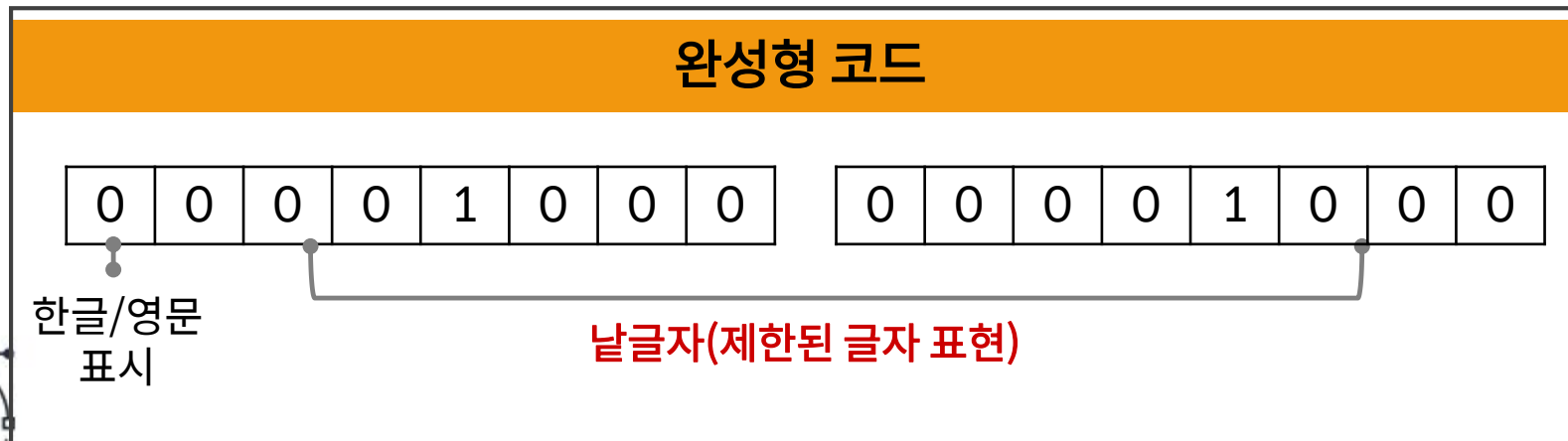
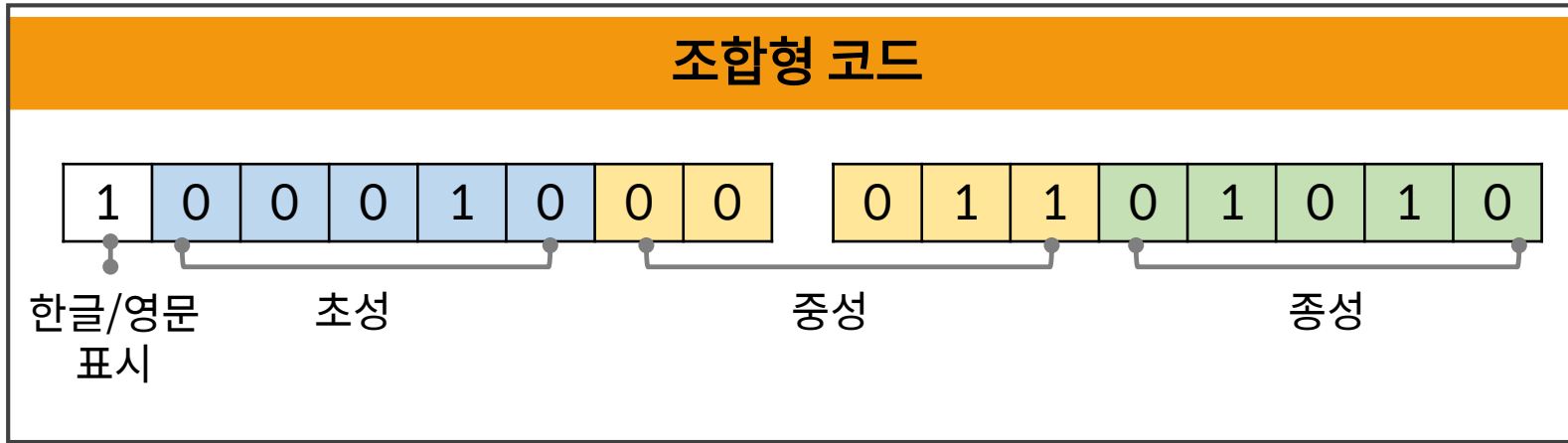


3) 텍스트의 표현



→ 한글 코드

■ 조합형 코드와 완성형 코드의 원리(키보드를 생각하면 안됨)



3) 텍스트의 표현



➔ 유니코드

- ▶ 각 나라별 코드 체계는 다양하고 복잡함
- ▶ 인터넷 환경에서는 모든 나라에서 **공통**으로 사용할 수 있는 코드 체계가 필요함
- ▶ 각 나라의 코드 체계를 **통합**하여 개발함
- ▶ 전 세계의 모든 문자를 8비트 단위인 **옥텟(Octet)**으로 표현함(**예전**)
- ▶ 기본적으로 하나의 문자를 **4개의 옥텟**으로 표현함(**예전**)

But

2개의 옥텟만을 사용하는 코드 세트가 정의되어 있어
주로 이 코드 체계를 사용함(**현재**)



4) 폰트



→ 폰트

폰트(Font)

인쇄 환경에서 사용하던 용어로 글자의 모양을 나타내며 **글꼴**이라고 부름

▶ 컴퓨터에서 사용되는 모든 문자의 **모양**과 **크기**에 대한 정보를 가지고 있음

구성 방법에 따른 구분

비트맵(**Bitmap**) 폰트
(픽셀)

벡터(**Vector**) 폰트
(좌표)



4) 폰트



→ 폰트의 속성

크기

포인트(Point)라는 단위로 부르며 pt로 크기를 나타냄

장평

한 글자의 가로 길이와 세로 길이의 비율

자간

글자와 글자 사이의 간격

MD이슈체	그래픽	느낌체	사랑체	파도체
HY중고딕	HY견고딕	HY신명조	HY전명조	HY그래픽M
HY궁서B	HY크리스탈M	HY백송B	HY헤드라인M	HY목각파임B
HY목판B	HY염서L	HY염서M	HY얇은샘M	HY올림도M
휴먼아미게	휴먼모음T	휴먼엑스포	휴먼매직체	휴먼2딕
문체부 바탕체	문체부 돌음체	문체부 제목 돌음체	문체부 제목 바탕체	각진제목체
가난한상체	굵은안상체	바탕한체	굵은바탕한체	가능바탕한체
굵은돌음한체	양재 소술	양재 튼튼B	양재 참숯B	양재 둥기

스마트시대의 멀티미디어 1234567890

스마트시대의 멀티미디어 1234567890

스마트시대의 멀티미디어 1234567890

스마트시대의 멀티미디어 1234567890

스마트시대의 멀티미디어 1234567890

스마트시대의 멀티미디어 1234567890

스마트시대의 멀티미디어 1234567890



[출처 : 한빛아카데미, 스마트 시대의 멀티미디어, 2019]

4) 폰트



➔ 고정간격 폰트 vs. 비례간격 폰트

모니터에 출력되는 형태에 따른 구분

고정간격 폰트

가로 길이가 **일정한** 폰트

비례간격 폰트

선택하는 **글꼴**에 따라 가로 길이가
일정하지 않은 폰트

구분	글꼴	H 글자의 경우
비례간격 폰트	바탕	HHHHHHH
	굴림	HHHHHHH
고정간격 폰트	바탕체	HHHHHH
	굴림체	HHHHHH



4) 폰트



→ 타입페이스

타입페이스(Typeface)

글씨를 써 놓은 모양을 나타내는 용어로 일반적으로 **글꼴**이라고 부름

사각형 글꼴

- **동일한** 넓이의 사각형 틀을 기본으로 함
- 대표 폰트: 맑은고딕체

비사각형 글꼴

- 사각형 틀을 **벗어난** 글씨체를 의미함
- 대표 폰트: 안상수체



4) 폰트



➔ 타이페이스의 종류와 정의

타입페이스	정의
세리프(Serif)	■ 문자의 끝부분을 뾰치게 만든 것 ■ 텍스트의 본문에 적합함 ■ 예 : Times, Bookman, Palatino
산세리프 (Sans Serif)	■ 글자의 끝부분에 뾰침이 없는 것 ■ 제목, 강조될 문장에 적합함 ■ 예 : Helvetica, Arial, Optima, Avant Garde(아방 가르드)
타입 스타일	■ 굵음(Boldface), 이탤릭(Italic) 등
타입 스타일 속성	■ 밑줄(Underlining), 외곽선(Outlining) 등
타입 크기	■ 일반적으로 포인트(pt) 단위로 표현함 - 1pt = 1/72인치 ■ 문자의 상단에서 하단까지의 거리를 말함 - 줄 간격을 위한 약간의 여백을 포함함



4) 폰트



→ 타이페이스



5) 비트맵 폰트와 벡터 폰트



➔ 비트맵 폰트

비트맵 폰트

2차원의 사각 평면을 작은 픽셀(Pixel) 단위로 분할하여 그 위에 글자나 이미지의 형상을 그대로 표현함

▶ 각 픽셀은 비트(Bit)에 해당하는 0 또는 1이 저장됨(계단 현상, RET 기술)

➡ 이러한 여러 개의 점들이 조합하여 정보가 표현됨

동근모꼴 공개 글씨체

Neo 동근모 글꼴

" 유명한 DOS용 비트맵 한글 글꼴을
트루타입 글꼴로 만나보세요."



5) 비트맵 폰트와 벡터 폰트



➔ 비트맵 이미지

비트맵 이미지

비트맵 방식으로 이미지를 저장하고 관리하는 것



GIF, JPG, PNG, BMP, TIFF, PCT, PCX 등

- ▶ 비트맵 폰트와 비트맵 이미지를 다른 용어로 래스터 폰트와 래스터 이미지라고도 함(Raster)



5) 비트맵 폰트와 벡터 폰트



➔ 비트맵 방식의 장·단점

장점

- 출력하기 위해 복잡한 연산을 거치지 않기 때문에 빠름
- 주로 화면 표시용으로 사용함(사진)

단점

- 글자나 이미지의 크기가 커지면 필요한 메모리 용량도 증가함
- 확대, 축소를 대비하여 가능한 많은 사이즈를 등록해 놓아야 하기 때문에 데이터 양이 증가함
- 글씨나 이미지를 확대하거나 축소하면 계단 현상이 발생하여 모자이크처럼 깨짐
➡ 앨리어싱(Aliasing) 현상



5) 비트맵 폰트와 벡터 폰트



➔ 앨리어싱 현상을 제거하기 위한 방법

- 1 모니터의 해상도를 높여 이미지를 좀 더 조밀하게 점으로 표현함
- 2 경계면의 밝기를 조절하여 중간 색조로 부드럽게 보이도록 처리하는 **안티앨리어싱(Anti-Aliasing)** 사용
- 3 계단현상을 제거하는 기술인 **RET(Resolution Enhancement Technology)** 사용



5) 비트맵 폰트와 벡터 폰트



➔ 벡터 폰트와 벡터 이미지

▶ 문자나 이미지의 모양을 윤곽선의 **방향**과 **길이**로 기억하는 방식(**좌표**)

시작하는
좌표의 위치(X1, Y1)지정

끝나는
좌표의 위치(X2, Y2) 지정

글자와
이미지 모양을
나타내기 위해



5) 비트맵 폰트와 벡터 폰트



➔ 벡터 방식의 장·단점

장점

- 크기나 해상도에 영향을 받지 않음
- 글자나 이미지를 확대하거나 축소해도 일그러짐 없이 깨끗한 모양을 유지함
- 글자나 이미지를 표현하기 위한 **좌표**만 저장하기 때문에 메모리 용량을 적게 차지함(**드로잉**)

단점

- 복잡한 수학적 알고리즘을 가지고 있기 때문에 크기에 상관없이 **알고리즘**을 저장할 메모리 공간이 필요함
- 글자 또는 이미지를 표현하는 **속도**는 비트맵에 비하여 떨어짐



Lesson 02

텍스트의 저장 형식

1) 메모장



- ▶ 1985년 마이크로소프트사의 윈도우 1.0부터 시작하여 현재까지 사용되고 있음

➡ 확장자는 *.txt 임

- ▶ 저장 문서에 대한 어떠한 정보나 포맷도 가지고 있지 않아 거의 대부분의 **문서 파일 포맷**에 대하여 편집이 가능함(**호환성**)

문서 파일 형식 중에서 가장 호환성이 높음



2) 워드프로세서(hwp, docx)



- ▶ 가장 많이 사용하는 것은 마이크로소프트사의 **MS워드**와 한글과컴퓨터사의 **한글임**
- ▶ 워드프로세서 파일의 포맷은 본문의 내용뿐만 아니라 폰트나 서식 정보 등도 함께 포함하고 있음(**형식**)
- ▶ 하나의 워드프로세서 파일을 다른 워드프로세서에서 사용하려면 해당 포맷으로 변경해야 함(**호환성이 낮음**)



2) 워드프로세서(hwp, docx)



➔ MS워드의 특징

- 1 확장자 *.docx로 저장됨
- 2 작성한 문서를 PDF 또는 XPS(XML Paper Specification) 형식으로 저장 가능함
- 3 MS 오피스의 워드, 엑셀, 파워포인트 등이 문서 표준을 공개하여 모든 워드프로세서 및 문서편집기에서 **호환**이 가능함(**오픈, 구글 오피스**)
- 4 전 세계적으로 시장 점유율이 높음



2) 워드프로세서(hwp, docx)



→ 한글의 특징

- 1 확장자 *.hwp로 저장됨
- 2 한글 워드프로세서의 시초는 1989년에 개발한 '아래아한글 1.0'으로 고어를 포함한 완벽한 한글을 구현함
- 3 작성한 문서를 PDF, XML 형식으로 저장이 가능함
- 4 MS 오피스가 점령하지 못한 유일한 나라가 될 정도로 토종 워드프로세서로서 성능과 안정성이 뛰어남(애국심 마케팅)



3) PDF(pdf)



- ▶ 어도비사(Adobe Systems)에서 만든 문서파일 포맷임
- ▶ 대부분의 운영체제에서 호환되어 읽기가 가능함(스마트폰)
- ▶ 확대나 축소를 해도 해상도가 변하지 않음
- ▶ 자체적인 압축 기능을 포함하고 있어 온라인 및 오프라인 환경에서 문서를 쉽게 공유할 수 있음(암호화, 암호 잠금)

“ 보안성이 높음 ”



3) PDF(pdf)



➔ 장 · 단점

장점

- 문서의 **호환성**이 높음
- 파일의 무결성을 가짐
- 문서의 사용 및 관리가 편리함
- 문서 보안이 우수함(**암호**)

Vs.

단점

- 문서의 제작 방식이 동일하지 않고 **다양한 파일 형식**이 존재함
- **편집**이 불편함
- 모니터에서 가독성이 낮음



4) 웹 기반 문서(html, xml)



→ HTML

HTML

인터넷에 웹사이트를 만들기 위한 프로그램 언어(Hyper Text Markup Language) (비순차, 태그, SGML)

확장자: *.htm 또는 *.html

- ▶ 1990년대 팀 버너스 리(Timothy John Berners-Lee)에 의해 창안됨
- ▶ 웹 문서를 작성하는 보편적인 방법으로 단순하고 사용하기 편리함(웹 서버, 웹 브라우저, HTTP)



4) 웹 기반 문서(html, xml)



→ HTML

- ▶ 브라우저를 통해 볼 수 있는 대부분의 웹페이지들은 **HTML**로 작성됨
- ▶ 브라우저상에 정보를 표시하기 위해 **마크업** 심볼의 집합으로 구성됨

특정 위치에 삽입되는 문자(명령어)나
기호를 의미하며 **태그**라고도 함(**HTML5**)



4) 웹 기반 문서(html, xml)



➔ 기본적인 HTML 태그

태그	설명	태그	설명
<HTML>...</HTML>	HTML 형식의 웹페이지를 선언	<MENU> ... </MENU>	리스트 아이템 처음과 끝 부분에 씀
<HEAD> ... </HEAD>	페이지의 헤드(Head)부를 지정		리스트 아이템 항목들
<TITLE> ... </TITLE>	페이지의 타이틀을 선언	
	줄을 바꿈
<BODY> ... </BODY>	페이지의 몸체(Body) 부분을 지정	<P>	문단을 바꿈
<HN> ... </HN>	...부분을 Hn 크기의 글자로 만들	<HR>	수평선을 그음
 ... 	...부분을 볼드체로 처리	<PRE> ... </PRE>	미리 지정된 형태의 텍스트
<I> ... </I>	...부분을 이탤릭체로 처리		이미지를 불러옴
 ... 	순서 없는 리스트 항목을 만들	...	하이퍼링크 지정
...	번호가 있는 리스트 항목을 만들		



4) 웹 기반 문서(html, xml)



➔ XML(eXtensible Markup Language)

HTML

제한된 개수의 태그를 사용하기 때문에 문서의 형태를
충분히 표현할 수 없음(태그 제한)

▶ HTML이 한계를 나타내면서 다양한 변종들이 등장함

- 마크업
 - 액티브엑스(ActiveX), 플래시(Flash), 스크립트(Script) 언어, DHTML(Dynamic HTML), 채널
- 브라우저 간의 **호환성** 문제 해결하지 못함



4) 웹 기반 문서(html, xml)



➔ XML(eXtensible Markup Language)

- ▶ HTML이 가지는 **상호 호환성 문제**를 해결하기 위해 XML이라는 새로운 마크업 언어를 표준화함(**SGML-XML**)
- ▶ HTML과 SGML의 장점을 기반으로 만들어진 웹페이지 기술 언어임

HTML

태그의 종류가
한정적임

XML

태그를 사용자가 **직접
정의**할 수 있고 그 태그를
다른 사람들도 사용 가능함



4) 웹 기반 문서(html, xml)



➔ XML의 특징

1 문서를 **내용, 구조, 서식**으로 개별적인 파일로 생성함(**메타 언어**)

2 문서를 구성하는 각 요소들의 **독립성**이 보장되기 때문에
문서의 호환성이 높고 내용이 독립적임

3 구성 요소별로 **저장, 검색, 재가공**이 용이함



4) 웹 기반 문서(html, xml)



➔ XML의 특징

4

확장성이 뛰어나 데이터베이스나 스프레드시트 등과 같은 **구조화된 데이터들**을 XML로 쉽게 변환할 수 있음

5

데이터를 화면에 출력하기 위해서는 화면 표현용 언어인 **XSL(eXtensible Stylesheet Language)**이 필요함

6

디지털 자료 보존에도 유용함



5) 스마트폰에서 지원하는 텍스트 파일 형식



1 스마트폰에서 문서 보기

- 텍스트, 웹페이지, 워드, 파워포인트, 엑셀, PDF 등은 별도의 애플리케이션 없이 iOS, 안드로이드 모두에서 미리 보기가 가능함
- 한글 문서는 안드로이드에서는 **미리 보기**가 가능함
BUT iOS에서는 폴라리스 오피스나 한컴오피스 등을 이용해야 미리 보기가 가능함



5) 스마트폰에서 지원하는 텍스트 파일 형식



2 스마트폰에서 문서 편집하기(불편함)

iOS

- iWorks(Pages, Keynote, Numbers), 폴라리스 오피스 같은 애플리케이션을 이용해 편집함(워드)

안드로이드

- 씽크프리(Thinkfree), 폴라리스 오피스 같은 애플리케이션을 이용해 편집함(워드)

- 한글 문서는 한컴오피스, 폴라리스 오피스, 씽크프리 등을 이용해 편집함

3 스마트폰에서 전자책 보기

- 어도비 뷰어(Adobe Viewer)를 사용하면 인디자인(Adobe InDesign) 프로그램으로 만든 전자책을 볼 수 있음



6) 문자 인식 기술



➔ 문자 인식 기술의 개요

문자 인식(Character Recognition) 기술

기존의 인쇄 자료나 손으로 기록한 문자, 기호, 마크 등을 컴퓨터가 자동으로 인식하는 기술



광학적인 장치를 사용하여 인식하기 때문에
광학 문자 인식(**OCR, Optical Character Recognition**)이라고도 함

▶ 패턴 인식(**Pattern Recognition**)의 한 종류임(**특징점**)

▶ 1970년대부터 상업적 용도로 널리 사용되기 시작함



6) 문자 인식 기술



➔ 문자 인식 기술의 개요

문자 인식 방법

패턴 정합(Pattern Matching)

- 문자의 유사성 및 정합도에 의해 문자를 식별하는 방식
- 주로 **인쇄 문자**의 인식에 사용됨

구조 분석(Structure Analysis)

- 문자 고유의 특징적인 선의 형태와 특성에 의해 문자를 식별하는 방식(**특징점**)
- 주로 **필기 문자**의 인식에 사용됨(**어려움**)



7) 문자 인식 장치와 문자 인식 소프트웨어



➔ 문자 인식 장치

문자 인식 장치

OCR 판독기와 소프트웨어로 문자를 판독하고 식별하여 컴퓨터가 이해할 수 있는 코드로 변환시키는 장치

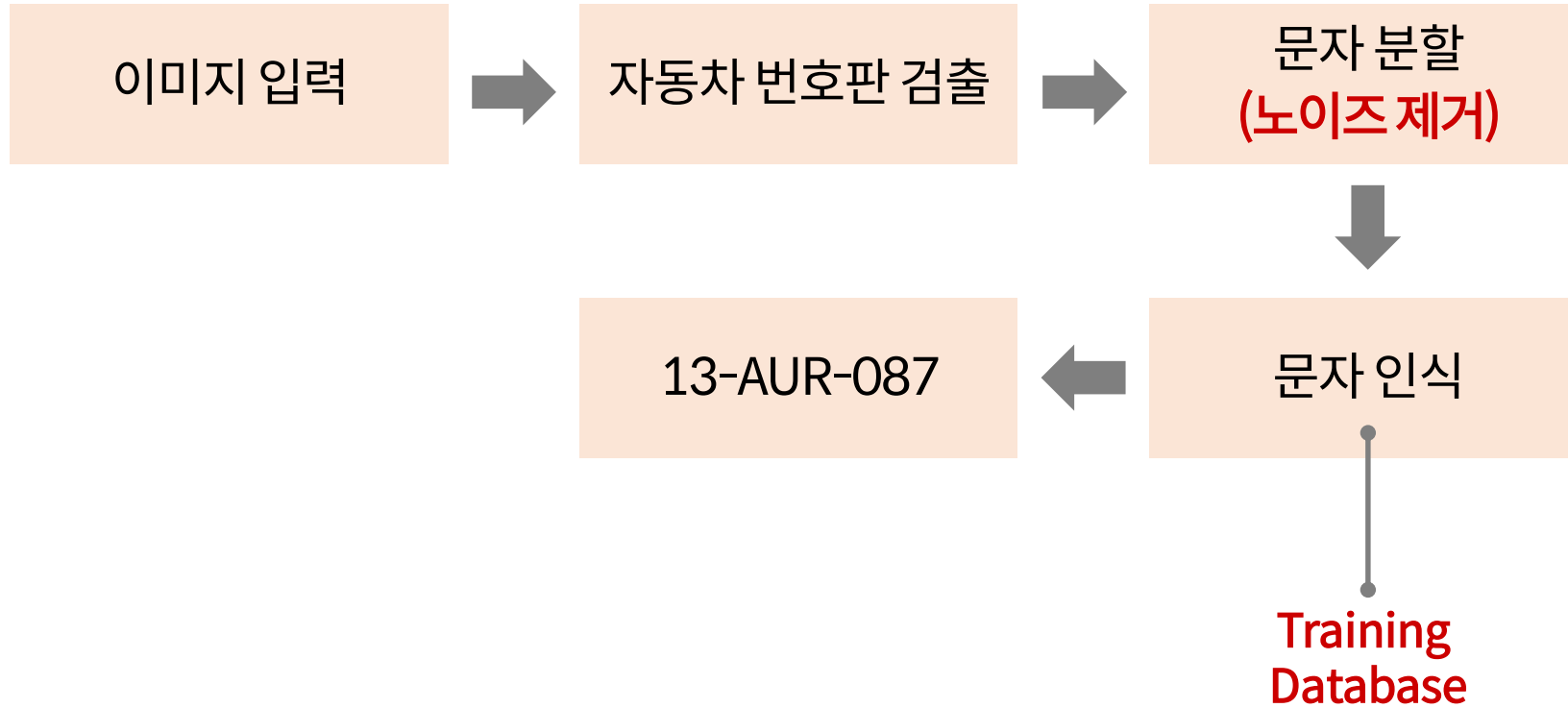
- ▶ 인식의 대상이 되는 문자는 보통 식별하기 유리한 형태로 **규격화**되어 있음
- ▶ 광전변환장치, 인식처리장치, 기억장치, 출력장치로 구성됨



7) 문자 인식 장치와 문자 인식 소프트웨어



➔ 문자 인식 장치



7) 문자 인식 장치와 문자 인식 소프트웨어



➔ 문자 인식 소프트웨어

▶ 다양한 환경의 요구에 의하여 **문자 인식 소프트웨어**가 다시 주목 받고 있음

➔ 전자책(E-book)과 모바일 등

대표적인 문자 인식 소프트웨어

어비(ABBY)

네이버 랩(Naver Lab)의 문자 인식 서비스

구글의 문서용 광학 문자 인식

어도비 아크로벳 PRO(Adobe Acrobat Pro)





1 필기체 문자를 인식하는 과정

- 문자의 변형 형태를 일정한 형태로 **정규화**하는 전처리 작업

기계 학습 알고리즘을 사용하여
문자의 특징을 추출하고 형태를 분류함(**특징점**)

문자나 단어의 사용 빈도, 언어의 형태학적 정보를
이용하여 문자를 인식함



7) 문자 인식 응용 기술



2 필기체 문자의 정확도를 향상시키기 위한 지표(제한)

- 문자의 특징적인 선을 명확하게 구별하여 씀
- 자연스러운 글자 형태를 사용함
- 문자 기입란의 크기와 모양을 동시에 제한하는데 여기에 맞추어 씀

3 C-Pen

- 아라미디어(Aramedia.com)에서 만든 손바닥 크기의 스캐너로 블루투스와 USB 연결이 가능함
- 스캔한 문자들을 내장된 OCR 기능을 사용하여 PC의 소프트웨어로 전송함(인식)



7) 문자 인식 응용 기술



4 모바일 문자 인식

- **문자 인식 애플리케이션**을 이용하여 카메라로 원하는 대상을 촬영한 후 영역을 지정하는 방식(**카드번호**)

5 휴대용 스캐너의 문자 인식

- 스캔과 동시에 이미지를 그림판, 포토샵, 한글 등에 삽입함



7) 문자 인식 응용 기술



6 스마트 기기와 웨어러블 기기에서 문자 인식

스마트 기기에서 문자 인식

갤럭시 노트 화면의
S펜 버튼을 누르면 나타나는
‘액션 메모’ 기능

웨어러블 기기에서 문자 인식

삼성의 갤럭시 기어와
구글의 구글 글래스에
탑재된 **광학 문자 인식** 기술



8) 전자책



종이책

전자책

- 배터리와 관계가 없음(오프라인)
- 눈이 덜 피곤하고 저렴함(아날로그)
- 전자책에 비해 훨씬 견고하고 콘텐츠의 종류도 훨씬 많음



8) 전자책



종이책

전자책

- 동영상을 담을 수 있음
- 전자책은 콘텐츠를 의미하는 **디지털 책(Digital Book)**과 **전자책 리더(Reader)**로 나뉨

디지털 책

CD-ROM과 같이 패키지 형태나 유무선 인터넷을 통해 유통됨(**스마트폰**)

전자책 리더

디지털 책을 읽을 수 있게 하는 하드웨어 또는 소프트웨어를 의미함



8) 전자책



➔ 전자책의 특징

1 콘텐츠와 콘텐츠를 전달하는 매체가 분리됨

- 하나의 단말기로 여러 개의 전자책을 볼 수 있음

BUT 콘텐츠를 각 단말기에 적합한 포맷으로 변환해야 함

2 **스크린 미디어**로 전환하면서 새로운 읽기 습관이 나타남

다중 읽기 (Multiple Reading)	텍스트로부터 이탈하는 몰입 대 조작 의 습관이 나타남(다중)
소셜 읽기 (Social Reading)	다른 독자와의 교류 가 텍스트의 세부적인 수준으로까지 심화됨
증강 읽기 (Augmented Reading)	아이트래킹 기술을 활용해 독자의 안구움직임을 추적 하고 낱말의 뜻을 알려줌



8) 전자책



➔ 전자책이 미디어 환경에 미치는 영향



읽기의 부활과 출판 산업의 활성화
(읽는 사람만 읽음)

데스크톱, 태블릿, 스마트폰의 역할이
기능적으로 **분업화** 됨(각자의 역할)

새로운 글쓰기 공간의 창출







Q

현 기술로 필기체를 인식 할 수 있을까요?





Q

현 기술로 필기체를 인식 할 수 있을까요?

A

현재의 기술로 인쇄체 인식은 잘되고 있습니다.

이유는 인쇄체의 경우 특정 패턴으로 글씨가 작성되어
특징 추출이 용이하여 인식이 잘되기 때문입니다.

즉, 인쇄체의 경우 변형이 아주 적어 오인식이 거의 되지 않습니다.

그러나 필기체의 경우는 이야기가 다릅니다.

전세계 모든 사람들은 각자만의 방식으로 필기를 합니다.

즉, 모든 사람들이 약간은 비슷할 수 있지만 모두 다른
필기체를 구사합니다.





Q

현 기술로 필기체를 인식 할 수 있을까요?

A

만약 필기체 인식을 만들려면 모든 사람의 특성을 고려한 시스템을 만들어야 하는데 이는 현실적으로 불가능합니다.

그러므로 현재의 기술로는 어려울 수 있고 인공지능과 빅데이터 기술을 접목하여 미래에는 가능할지 모릅니다.

필기체 인식 기술이 개발되면 사람들은 더 이상 타이핑 할 필요 없이 음성이나 텍스트로 컴퓨터를 조정할 수 있게 됩니다.

